

CSCI 6360 Parallel Computing

Assignment 3: 1D Stencil MPI

Quinten Pienaar

662041924

pienaq@rpi.edu

Due: March 16, 2026

1 Overview

The study investigates the performance of a one-dimensional stencil application implemented with the Message Passing Interface. The primary objective involved conducting strong and weak scaling experiments to evaluate the efficiency of the parallel algorithm across different compute environments. Observations indicate that execution time correlates directly with the array size and inversely with the number of compute nodes. While the MPI implementation achieves significant speedups compared to a serial baseline, it remains slower than specialized multi-gpu implementations implemented in previous assignments.

2 Testing

Experimental evaluations were performed across configurations involving one, two, and four compute nodes. The computational environment utilized IBM POWER9 CPUs, and the implementation leveraged the C programming language MPI API. A bash script was employed to automate the testing suite and ensure consistency across the fifteen experimental trials. This script managed the allocation of resources and the systematic variation of ranks and array sizes.

Performance was quantified using the `getticks` function, which provides high-resolution timing by reading the CPU cycle counter at a frequency of 512,000,000 ticks per second. This timing window encompasses the entire parallel workflow, including data scattering, non-blocking communication for halo exchanges, stencil calculations, and the final data gathering process.

3 Implementation

The parallel architecture utilizes an MPI barrier to ensure synchronization across all processes before the primary execution and timing phases begin. Rank zero serves as the coordinator by initializing the input array to a uniform value of one and distributing data partitions to all participating ranks via the scatter primitive. Each rank allocates a local buffer with additional space to accommodate halo regions of size H on either side of the local data chunk.

The implementation manages halo exchanges by populating an array of MPI Request objects to track up to four distinct communication operations. Specifically, the code issues non-blocking receives before non-blocking sends to mitigate potential deadlocks and facilitate overlapping communication with computation. A synchronization point is established using the MPI Waitall primitive, which ensures that all neighbor data is successfully transferred into the local halo buffers before the stencil calculation begins. Upon completion of the local computation, the results are collected back to the master rank through a gather operation for final assembly and verification against the serial baseline.

3.1 Validation

Validation of the numerical output involves a sequential traversal of the final array to compare computed values against analytical expectations. For an input array initialized entirely to one, the expected output at any given index depends on its proximity to the boundaries. The expected output at index i for an array of size N with halo radius H is defined as follows:

$$y[i] = \begin{cases} 1 + H + i & \text{if } i < H \\ 2H + 1 & \text{if } H \leq i < N - H \\ N - i + H & \text{if } i \geq N - H \end{cases} \quad (1)$$

Interior elements accumulate a uniform value of $2H + 1 = 129$ for a halo radius of $H = 64$. Elements located within the boundary regions accumulate lower values because the stencil window extends beyond the defined array bounds. The expected values in these regions exhibit a linear taper toward the edges of the grid.

4 Experimental Results

Table 1 presents the execution time in seconds for each combination of node, rank, and array size for the strong scaling experiments. Table 2 provides the corresponding data for the weak scaling trials. The visual representation of these performance metrics is illustrated in Figure 1.

Table 1: Strong scaling results for $N = 2^{30}$ and a HaloSize of 64

Ranks	Nodes	Total elements	Time (s)	Speedup	Status
1 (serial)	1	2^{30}	930.37	$1.00\times$	SUCCESS
2	1	2^{30}	466.46	$2.00\times$	SUCCESS
4	1	2^{30}	234.24	$3.97\times$	SUCCESS
8	1	2^{30}	118.40	$7.86\times$	SUCCESS
16	1	2^{30}	60.26	$15.44\times$	SUCCESS
32	1	2^{30}	31.32	$29.70\times$	SUCCESS
64	2	2^{30}	16.32	$57.01\times$	SUCCESS
128	4	2^{30}	8.59	$108.31\times$	SUCCESS

Table 2: Weak scaling results for 2^{24} elements per rank and a HaloSize of 64

Ranks	Nodes	Total elements	Time (s)	Status
2	1	2^{25}	14.59	SUCCESS
4	1	2^{26}	14.66	SUCCESS
8	1	2^{27}	14.78	SUCCESS
16	1	2^{28}	15.04	SUCCESS
32	1	2^{29}	15.66	SUCCESS
64	2	2^{30}	16.34	SUCCESS
128	4	2^{31}	17.11	SUCCESS

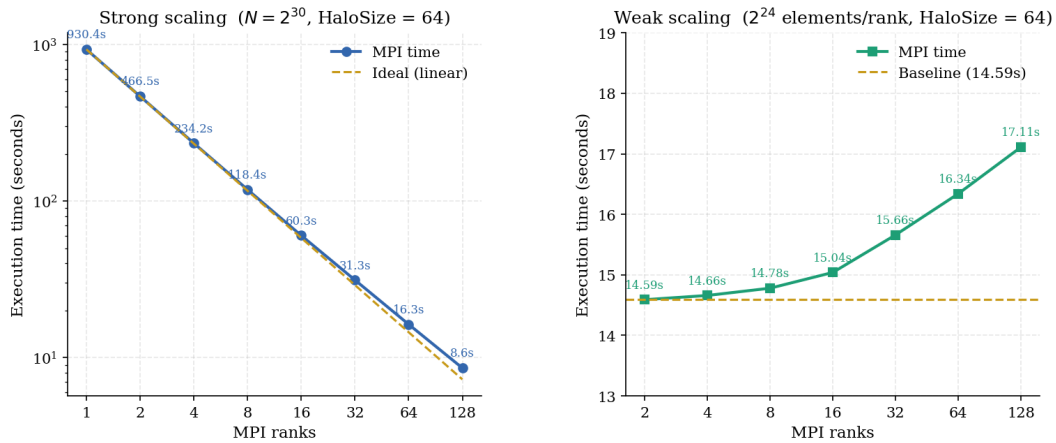


Figure 1: 1D Stencil execution time versus grid size for varying MPI ranks.

5 Analysis

The experimental results generally align with theoretical scaling models. Analysis of the strong scaling data reveals that execution times for high rank counts are slightly higher than ideal projections. This divergence is likely attributable to the increased ratio of communication overhead to local computation as the workload per rank decreases. As the computational task per node shrinks, the time spent on network latency for halo exchanges becomes a more significant portion of the total execution time.

Ideally, weak scaling results should exhibit a constant execution time as the total problem size increases proportionally to the number of ranks. The experimental data, however, reveals a deviation from this ideal behavior, characterized by an initial phase of sub-linear growth followed by a more pronounced linear trend in execution time.

The initial sub-linear phase occurs when all MPI ranks are situated on a single compute node. During this stage, the overhead associated with halo exchanges is minimal because the processes utilize high-speed intra-node communication pathways rather than external network interfaces. Although the execution time increases slightly as the number of ranks grows, the rate of increase remains low because the communication overhead grows more slowly than the cumulative computational power of the node.

The transition to a linear growth trend becomes apparent as the system scales to multiple nodes and larger global array sizes. This behavior is primarily driven by the centralized architecture of the implementation, where rank zero is responsible for allocating, initializing, and distributing the entire global array. Because the total number of elements N grows linearly with the number of ranks in a weak scaling study, the time required for rank zero to perform the scatter and gather operations also increases linearly. This serial bottleneck at the master rank eventually becomes the dominant factor in the total execution time, overshadowing the constant local computation performed by each individual rank.

Despite these scaling overheads, the MPI implementation provides substantial performance gains over the sequential baseline, achieving speedups as high as 108 times when utilizing 128 ranks. However, the performance remains significantly lower than specialized multi-GPU implementations from previous assignments which completed stencil calculations in as low as 0.28 seconds. This performance gap highlights the inherent limitations of CPU-based message passing compared to the massive parallelism and superior memory bandwidth provided by modern GPUs.

6 Conclusions

This assignment demonstrates the effectiveness of MPI for distributing stencil computations across multiple CPU nodes. The results confirm that while parallelization offers massive speedups over sequential code, the overhead of message passing must be carefully managed to maintain efficiency at scale.

MPI remains a versatile choice for large-scale distributed memory systems where GPU resources are limited or when the problem size exceeds the memory capacity of a single accelerator. Future research could investigate the implementation of decentralized data management using MPI-IO to eliminate the serial bottleneck caused by the master rank managing the global array.